

TACM / PRNG Structural Diagnostics

Empirical Measures, Seed-Conditioned Variance, Null Calibration and Reproducibility
Boundaries

Pascal Montmory

May 2026

Scope and evidentiary policy. This manuscript presents the TACM / PRNG structural diagnostics framework as a self-contained contribution. It defines the diagnostic objects, states the non-claims, develops empirical-measure and seed-conditioned Monte Carlo diagnostics, introduces a null-calibrated variance test, and reports a reproducible CPU-only baseline. Claims are explicitly labelled as definitions, reproducible evidence, reported evidence requiring archival raw logs for independent verification, or stated limitations.

Abstract

This manuscript introduces TACM, a structural diagnostic framework for pseudo-random number generators under finite Monte Carlo workloads. TACM is formulated around empirical measures of PRNG output, seed-conditioned replicated Monte Carlo estimators, null-calibrated variance diagnostics, ECDF interpretation and exact rare-event calibration.

The central position is that TACM does not replace established PRNG batteries such as TestU01 or BigCrush. Instead, it adds a Monte Carlo workload-facing layer: it measures how seed-conditioned PRNG streams affect replicated Monte Carlo estimators. The core calibrated diagnostic is

$$Z_s(f) = F_{\chi^2_{R-1}} \left(\frac{(R-1)V_s(f)}{\sigma_f^2/N} \right),$$

where $V_s(f)$ is the empirical variance of replicated Monte Carlo estimates under seed s .

The verified CPU-only baseline in this document tests four public NumPy bit generators, four integrands, $S = 1000$ seeds, $R = 8$ replicates and $N = 50\,000$ draws per replicate. It shows that large raw max/min variance ratios occur naturally, but after null calibration the tested public PRNGs do not show strong tail-pathological behaviour on these simple workloads. Several generator/integrand pairs show mild calibration drift; none clearly survives a strict Bonferroni correction at $m = 16$, and exact binomial rare-event PIT checks are compatible for all four public generators.

Reported TestU01 / BigCrush results and cross-backend Montmory_CTACM observations are included as reported evidence requiring archival raw logs and stream hashes for independent verification. They are not treated as independently reproducible evidence in the present manuscript.

Contents

1	Scientific Positioning	3
1.1	What TACM is	3
1.2	What TACM is not	3
1.3	What is new in the consolidated version	3
2	Evidentiary Status of Claims	3
3	PRNG Model and Seed Model	4
3.1	Generator model	4
3.2	Seed model	4
3.3	Observables	5
4	Empirical Measures of PRNG Output	5
5	Classical and Projected Diagnostics	5
5.1	Marginal coverage	5
5.2	Projected transition diagnostics	6
6	Seed-Conditioned Monte Carlo Variance	6
7	Null-Calibrated Diagnostic	6
7.1	Tail-rate diagnostic	7
7.2	Three-class reading	7
8	Rare-Event Exact Calibration	7
9	Reproducible Public CPU Baseline	7
9.1	Protocol	7
9.2	Calibrated variance results	8
9.3	Exact binomial rare-event PIT results	8
10	Reported Montmory_CTACM Evidence	8
10.1	Reported TestU01 summary	9
10.2	Reported earlier inter-seed dispersion	9
11	Figures	9
11.1	ECDF of calibrated Z_s	9
11.2	Histograms	15
12	Reproducibility Manifest	17
12.1	Public baseline bundle	17
12.2	Independent-verification requirements for reported CTACM results	17
13	Limitations	18
14	Conclusion	18
15	Future Work	18

1 Scientific Positioning

1.1 What TACM is

TACM is a structural diagnostic framework for PRNG behaviour under finite Monte Carlo workloads. It studies objects of the form

$$\Phi(G, p, s, N),$$

where G is a deterministic generator, p a parameter setting, s a seed, N a finite horizon, and Φ a declared observable such as a Monte Carlo estimator, empirical transition matrix, empirical measure or cross-backend replay hash.

The practical question is:

How much does the seed affect Monte Carlo outputs under a declared workload?

1.2 What TACM is not

TACM is not:

- a replacement for TestU01, PractRand, BigCrush or other stream-level batteries;
- a proof of randomness;
- a proof of cryptographic security;
- a theorem about all seeds, all horizons or all workloads unless such a theorem is explicitly provided;
- a validation of any proprietary generator merely from summary tables.

The strict non-claim is:

Empirical randomness tests are evidence, not proof.

1.3 What is new in the consolidated version

Compared with the initial v1.0 document, this consolidated version adds:

- the empirical-measure formulation of PRNG output;
- a careful separation between raw ratios and calibrated evidence;
- the null-calibrated diagnostic $Z_s(f)$;
- ECDF and tail-rate interpretation;
- exact binomial rare-event PIT checks;
- a reproducible CPU-only public-generator baseline with $S = 1000$ seeds;
- explicit evidence-status labels for all non-reproducible reported results.

2 Evidentiary Status of Claims

Table 1: Evidentiary status of claims used in this manuscript.

Item	Content	Status
Definitions	PRNG tuple, seed model, observables, empirical measures, replicated variance	Verified definitions.
Formulae	χ^2 , projected transition diagnostics, $V_s(f)$, ρ_f , $Z_s(f)$, Q_{tail}	Mathematical definitions / diagnostic formulas.

Item	Content	Status
Public CPU baseline	MT19937, PCG64, Philox, SFC64; $S = 1000$, $R = 8$, $N = 50\,000$	Reproducible local evidence; CSV, scripts and figures available.
Exact binomial rare-event PIT	Randomized binomial PIT for $1_{\{u>0.99\}}$ reconstructed from estimates	Reproducible local evidence; summary CSV generated.
Montmory_CTACM BigCrush	Reported 160/160 on x86, Metal and CUDA	Reported evidence; raw logs required for independent verification.
Montmory_CTACM cross-backend replay	Reported bit-exact CPU/Metal/CUDA single-lane equality	Reported evidence; stream hashes required for independent verification.
Earlier inter-seed dispersion table	$N = 10^6$, $R = 8$, 100 seeds, multiple PRNGs	Reported evidence; raw tables required for independent verification.
Power tests	injected marginal bias, temporal correlation, rare-event deletion	Future work unless commands and outputs are added.
Security claims	unpredictability, cryptographic strength, adversarial resistance	Not claimed.

3 PRNG Model and Seed Model

3.1 Generator model

A deterministic PRNG instance is represented as

$$G = (S, P, A, T, O, U),$$

where S is the state space, P the parameter space, A the output alphabet, $T_p : S \rightarrow S$ the state transition, $O_p : S \rightarrow A$ the output map, and $U : A \rightarrow [0, 1)$ the map from output words to Monte Carlo uniforms.

Given initial seed s_0 and parameter value p ,

$$s_{t+1} = T_p(s_t), \quad x_t = O_p(s_t), \quad u_t = U(x_t).$$

For a finite horizon n , write

$$X_n(p, s) = (x_0, \dots, x_{n-1}), \quad U_n(p, s) = (u_0, \dots, u_{n-1}).$$

All empirical claims in this document are finite-window claims unless a theorem explicitly states otherwise.

3.2 Seed model

A seed model is a probability law or finite test set over a declared seed domain, together with an initialization map

$$\text{Init}_p : \text{Seed} \rightarrow S.$$

The caller seed and the effective internal state need not be identical.

Seed perturbation rules can be expressed as

$$N_\delta : \text{Seed} \rightarrow \mathcal{P}(\text{Seed}).$$

Examples include one-bit flips, bounded Hamming perturbations and modular additive neighbourhoods:

$$N_{1\text{bit}}(s) = \{s \oplus 2^j : 0 \leq j < w\},$$

$$N_{\text{add},k}(s) = \{(s + r) \bmod 2^w : -k \leq r \leq k\}.$$

3.3 Observables

An observable is a declared function on a finite word or uniform window:

$$\Phi : A^n \rightarrow \mathbb{R}^d, \quad \Phi : [0, 1)^n \rightarrow \mathbb{R}^d.$$

Examples include bit frequencies, transition matrices, TestU01 scores, empirical measures, back-end hashes and Monte Carlo estimates.

A seed-impact claim must specify:

- generator and parameters;
- seed model;
- perturbation rule or seed set;
- output horizon;
- observable;
- proof, exact enumeration, or reproducible empirical protocol.

4 Empirical Measures of PRNG Output

For a seed s , backend b and horizon N , the marginal empirical measure is

$$\mu_{s,b,N}^{(1)} = \frac{1}{N} \sum_{t=1}^N \delta_{u_{s,b,t}}.$$

The ideal marginal target is the Lebesgue measure λ on $[0, 1)$.

Temporal dependence is represented by block empirical measures:

$$\mu_{s,b,N}^{(k)} = \frac{1}{N - k + 1} \sum_{t=1}^{N-k+1} \delta_{(u_{s,b,t}, \dots, u_{s,b,t+k-1})}, \quad k \geq 2.$$

The ideal target is $\lambda^{\otimes k}$.

The structural principle is:

A useful PRNG for Monte Carlo should produce empirical measures stable across seeds, backends and workloads.

5 Classical and Projected Diagnostics

5.1 Marginal coverage

Partition $[0, 1)$ into m bins $I_1, \dots, I_m$. For seed s :

$$O_i(s) = \#\{t : u_{s,t} \in I_i\}, \quad E_i = \frac{N}{m}.$$

The χ^2 statistic is

$$\chi_s^2 = \sum_{i=1}^m \frac{(O_i(s) - E_i)^2}{E_i}, \quad \chi_{\text{norm}}^2(s) = \frac{\chi_s^2}{m - 1}.$$

This measures marginal bin coverage, not temporal mixing.

5.2 Projected transition diagnostics

Define a quantizer

$$Q_q : [0, 1) \rightarrow \{1, \dots, q\}, \quad b_t = Q_q(u_t).$$

The row-stochastic empirical transition matrix is

$$P_s(i, j) = \frac{\#\{t : b_t = i, b_{t+1} = j\}}{\#\{t : b_t = i\}}.$$

This is a finite Markov projection of the observed stream, not the internal dynamics of the PRNG.

If

$$1 = \lambda_1, \lambda_2, \dots, \lambda_q$$

are eigenvalues of P_s , define

$$\text{Gap}_s = 1 - |\lambda_2(P_s)|.$$

This projected gap is a diagnostic of finite-state mixing in the chosen quantization.

6 Seed-Conditioned Monte Carlo Variance

For an integrand $f : [0, 1) \rightarrow \mathbb{R}$, seed s , replicate r and sample size N :

$$I_{s,r}(f) = \frac{1}{N} \sum_{t=1}^N f(u_{s,r,t}).$$

Across R replicates,

$$\bar{I}_s(f) = \frac{1}{R} \sum_{r=1}^R I_{s,r}(f),$$

and

$$V_s(f) = \frac{1}{R-1} \sum_{r=1}^R \left(I_{s,r}(f) - \bar{I}_s(f) \right)^2.$$

Convention. $V_s(f)$ is the variance of replicated Monte Carlo estimates. It is not the within-run sample variance of the raw values $f(u_t)$.

The raw inter-seed ratio is

$$\rho_f = \frac{\max_s V_s(f)}{\min_{s, V_s(f) > 0} V_s(f)}.$$

It is an alert, not a calibrated conclusion.

7 Null-Calibrated Diagnostic

Under the null model,

$$I_{s,r}(f) \approx \mathcal{N} \left(\mu_f, \frac{\sigma_f^2}{N} \right).$$

Then

$$T_s(f) = \frac{(R-1)V_s(f)}{\sigma_f^2/N} \sim \chi_{R-1}^2.$$

Define

$$Z_s(f) = F_{\chi_{R-1}^2}(T_s(f)).$$

Under the null:

$$Z_s(f) \sim \text{Uniform}[0, 1].$$

The v1.3 interpretation is:

$$\rho_f \text{ high} \Rightarrow \text{raw alert}$$

but

$$Z_s(f) \not\sim \text{Uniform}[0, 1] \Rightarrow \text{calibrated diagnostic signal.}$$

7.1 Tail-rate diagnostic

Define

$$Q_{\text{tail}}(f) = \frac{1}{|\mathcal{B}|} \#\{s : Z_s(f) < 0.01 \text{ or } Z_s(f) > 0.99\}.$$

Under the uniform null:

$$\mathbb{E}[Q_{\text{tail}}(f)] \approx 0.02.$$

7.2 Three-class reading

$$\text{compatible,} \quad \text{calibration_drift,} \quad \text{tail_anomaly.}$$

- **compatible:** $p_{\text{KS}} \geq 0.05$ and $Q_{\text{tail}} \approx 0.02$.
- **calibration_drift:** $p_{\text{KS}} < 0.05$ but $Q_{\text{tail}} \approx 0.02$.
- **tail_anomaly:** $Q_{\text{tail}} \gg 0.02$ or a visible ECDF deformation concentrated near extremes.

8 Rare-Event Exact Calibration

For the rare-event indicator

$$f(u) = \mathbf{1}_{\{u > 0.99\}},$$

the count per replicate is

$$K_{s,r} = \sum_{t=1}^N \mathbf{1}_{\{u_{s,r,t} > 0.99\}}.$$

Under the null:

$$K_{s,r} \sim \text{Binomial}(N, p), \quad p = 0.01.$$

Because the binomial law is discrete, the randomized PIT is

$$Z_{s,r}^{\text{bin,rand}} = F_{\text{Binomial}(N,p)}(K_{s,r} - 1) + U_{s,r} \mathbb{P}(K = K_{s,r}),$$

with $U_{s,r} \sim \text{Uniform}[0, 1]$.

This provides an exact rare-event check that does not rely only on the Gaussian approximation.

9 Reproducible Public CPU Baseline

9.1 Protocol

The verified CPU-only baseline uses:

$$S = 1000, \quad R = 8, \quad N = 50\,000.$$

Generators:

$$\text{MT19937,} \quad \text{PCG64,} \quad \text{Philox,} \quad \text{SFC64.}$$

Integrands:

$$u, \quad \sin(2\pi u), \quad u^2, \quad \mathbf{1}_{\{u > 0.99\}}.$$

9.2 Calibrated variance results

Table 2: CPU-only public-generator baseline, $S = 1000$, $R = 8$, $N = 50\,000$.

Generator	Backend	Integrand	$\rho_{\max}/\min$	$\rho_{95/5}$	KS stat	p_{KS}	Q_{tail}	v1.3 verdict
MT19937	cpu	identity	40.522	7.287	0.0453	0.0322	0.026	calibration_drift
MT19937	cpu	quadratic	40.581	6.693	0.0235	0.6291	0.031	compatible
MT19937	cpu	rare_099	26.759	5.903	0.0382	0.1060	0.015	compatible
MT19937	cpu	sin	32.541	6.215	0.0291	0.3598	0.018	compatible
PCG64	cpu	identity	43.267	6.472	0.0472	0.0226	0.015	calibration_drift
PCG64	cpu	quadratic	34.453	6.701	0.0510	0.0107	0.016	calibration_drift
PCG64	cpu	rare_099	41.566	7.069	0.0205	0.7852	0.025	compatible
PCG64	cpu	sin	46.430	6.624	0.0210	0.7615	0.013	compatible
Philox	cpu	identity	48.845	6.278	0.0231	0.6494	0.013	compatible
Philox	cpu	quadratic	87.380	6.691	0.0140	0.9879	0.024	compatible
Philox	cpu	rare_099	73.519	7.333	0.0529	0.0071	0.022	calibration_drift
Philox	cpu	sin	22.635	6.058	0.0184	0.8805	0.012	compatible
SFC64	cpu	identity	31.403	6.183	0.0161	0.9537	0.019	compatible
SFC64	cpu	quadratic	33.220	6.440	0.0380	0.1091	0.018	compatible
SFC64	cpu	rare_099	73.639	6.125	0.0565	0.0032	0.018	calibration_drift
SFC64	cpu	sin	30.664	5.726	0.0247	0.5660	0.019	compatible

There are $m = 16$ KS tests. A strict Bonferroni threshold is

$$\frac{0.05}{16} = 0.003125.$$

The smallest observed KS p-value is approximately 0.003236, for SFC64 / rare_099, just above the strict threshold. Therefore none of the current KS rejections clearly survives a strict Bonferroni correction.

The baseline shows mild calibration drift, but no strong tail anomaly.

9.3 Exact binomial rare-event PIT results

The rare-event exact calibration uses 8000 replicate-level counts per generator.

Table 3: Exact binomial rare-event calibration, randomized PIT, $N = 50\,000$, $p = 0.01$.

Generator	Backend	Replicates	Mean K	KS stat	p_{KS}	Tail rate	Verdict
MT19937	cpu	8000	500.014	0.00806	0.6735	0.0209	compatible
PCG64	cpu	8000	499.989	0.00500	0.9878	0.0225	compatible
Philox	cpu	8000	499.708	0.01029	0.3629	0.0191	compatible
SFC64	cpu	8000	499.869	0.00982	0.4211	0.0169	compatible

This exact rare-event check supports the interpretation that the $1_{\{u > 0.99\}}$ baseline does not show a strong rare-event tail anomaly for the tested public generators.

10 Reported Montmory_CTACM Evidence

This section preserves the important v1.0 material but labels it correctly.

10.1 Reported TestU01 summary

Table 4: Reported TestU01 outcomes for Montmory_CTACM. Raw logs are required for independent verification.

Backend	SmallCrush	BigCrush	Status
x86 CPU	pass	160 / 160	Reported; raw log required
Apple Metal GPU	pass	160 / 160	Reported; raw log required
CUDA GPU	pass	160 / 160	Reported; raw log required

These outcomes must not be treated as independently reproducible repository evidence until the complete logs, TestU01 version, stream adapter, backend implementation hashes, compiler flags and SHA-256 log hashes are imported.

10.2 Reported earlier inter-seed dispersion

Table 5: Reported earlier structural dispersion, $N = 10^6$, $R = 8$, 100 seeds. Raw tables are required for independent verification.

PRNG	Backend	ρ_{u^2}	$\rho_{1_{\{u>0.99\}}}$	Seeds	Status
Montmory_CTACM	CPU	~ 10	~ 3.4	100	Reported
Montmory_CTACM	GPU lane 1	~ 10	~ 3.4	100	Reported
MT19937	CPU	~ 10	~ 14	100	Reported
MT19937-64	CPU	~ 1.9	~ 25	100	Reported
PCG32	CPU	~ 15	~ 3.2	100	Reported
xoshiro256**	CPU	~ 5.9	~ 10.5	100	Reported
Philox4x32-10	CPU	~ 4.6	~ 11	100	Reported
MRG32k3a	CPU	~ 7.7	~ 3.9	100	Reported

This table is useful historically, but v1.3 changes its interpretation: raw ratios motivate calibration, but do not by themselves prove seed pathology.

11 Figures

11.1 ECDF of calibrated Z_s

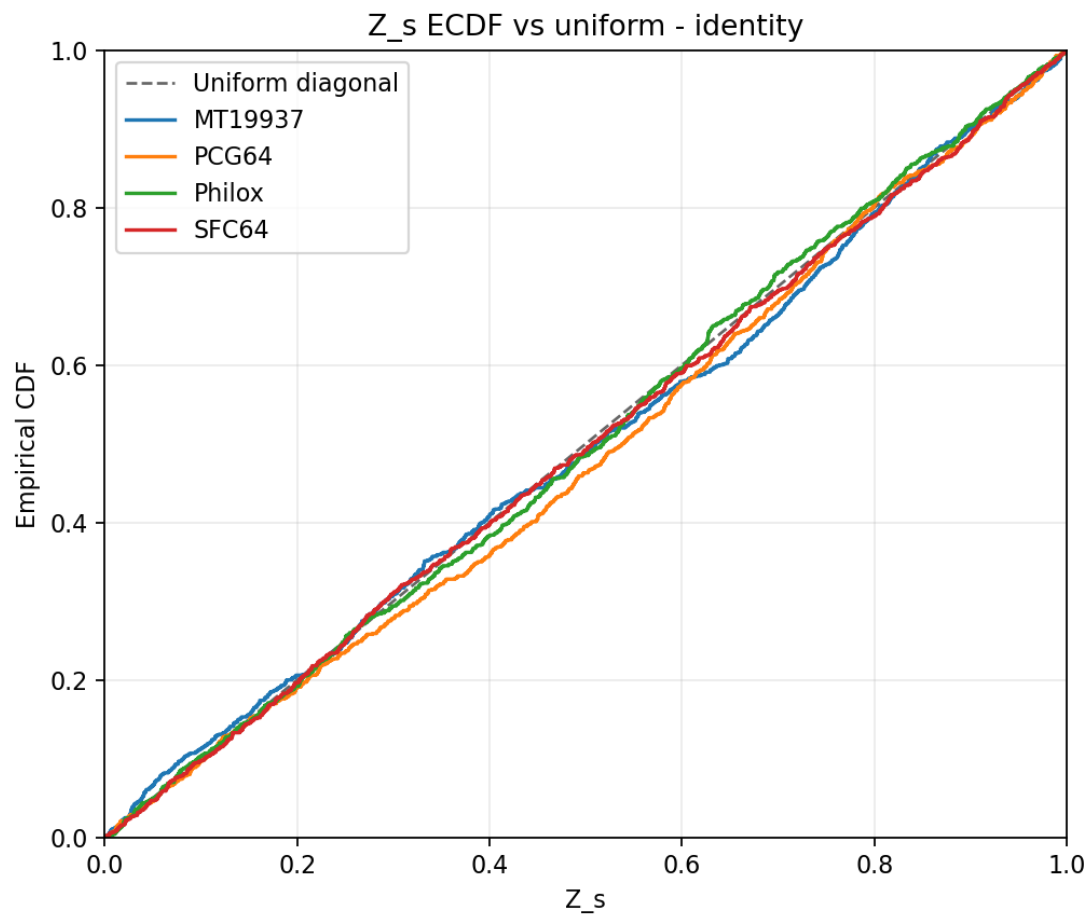


Figure 1: ECDF of Z_s for the identity integrand.

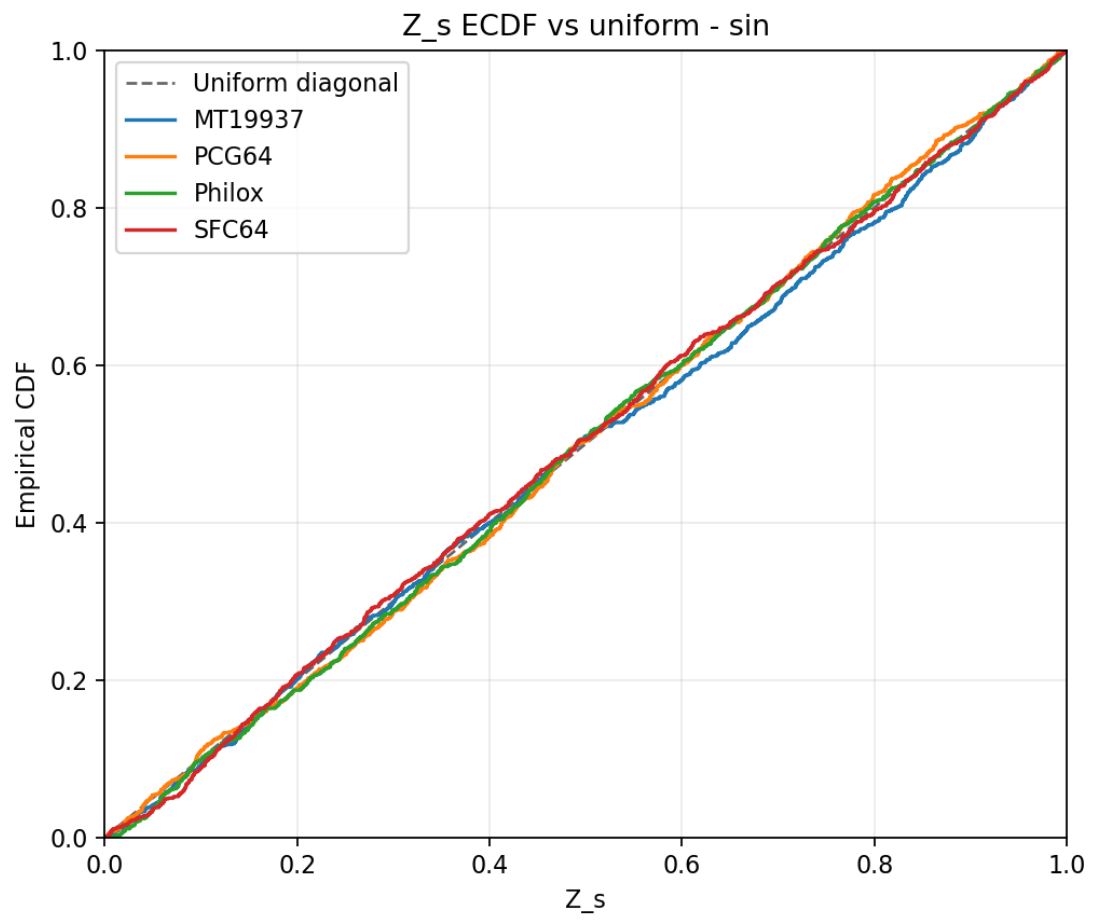


Figure 2: ECDF of Z_s for the sine integrand.

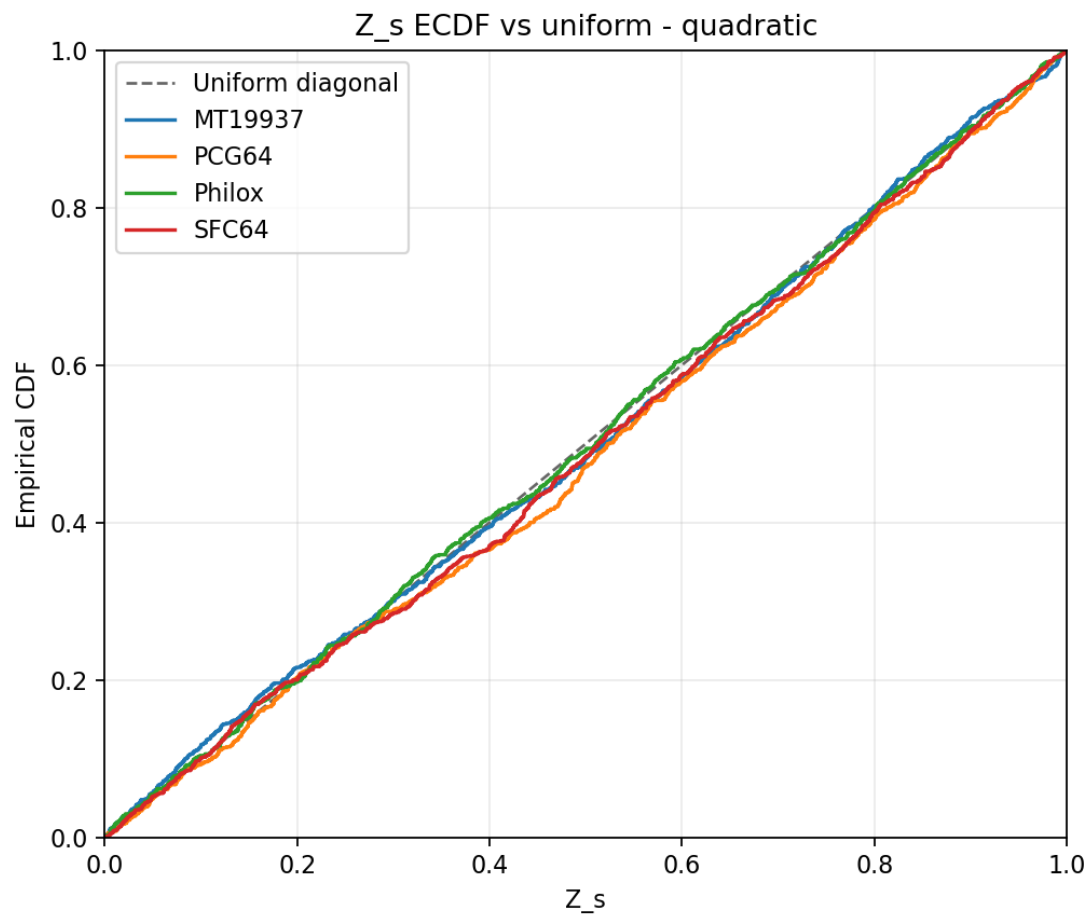


Figure 3: ECDF of Z_s for the quadratic integrand.

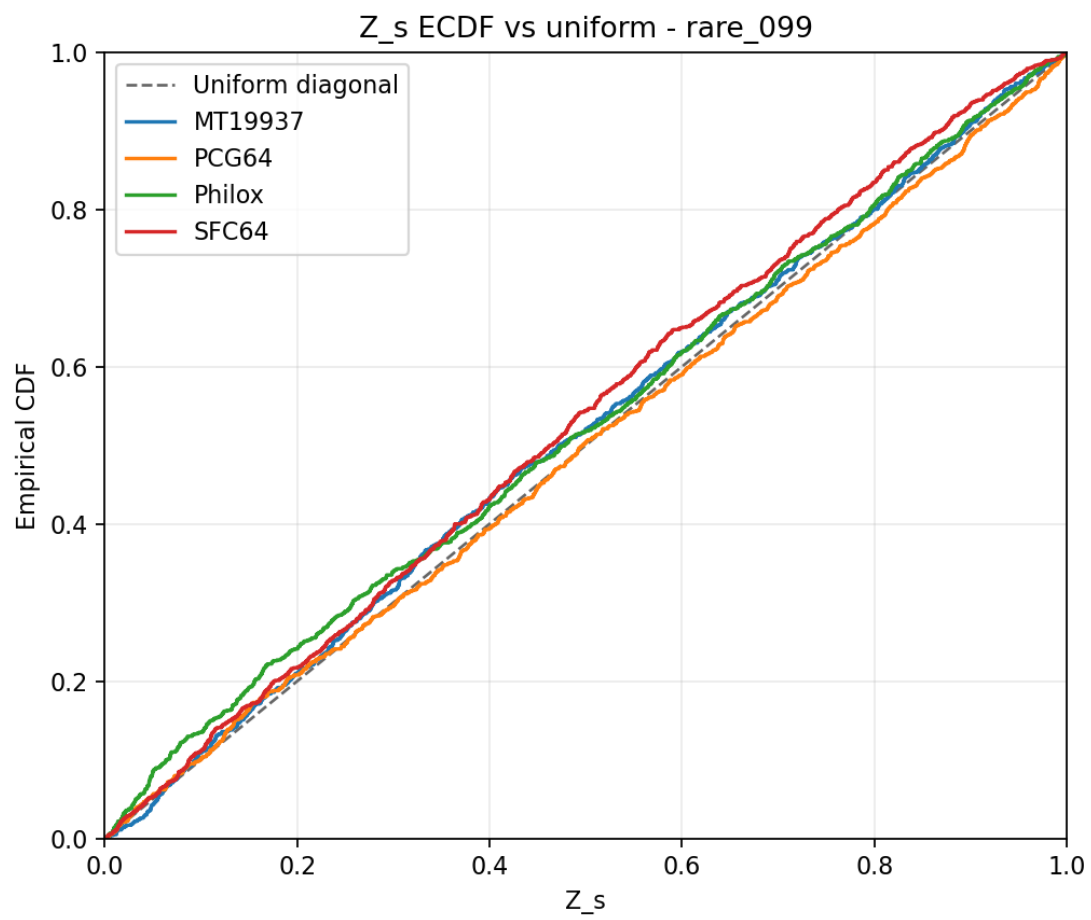


Figure 4: ECDF of Z_s for the rare-event integrand $1_{\{u>0.99\}}$.

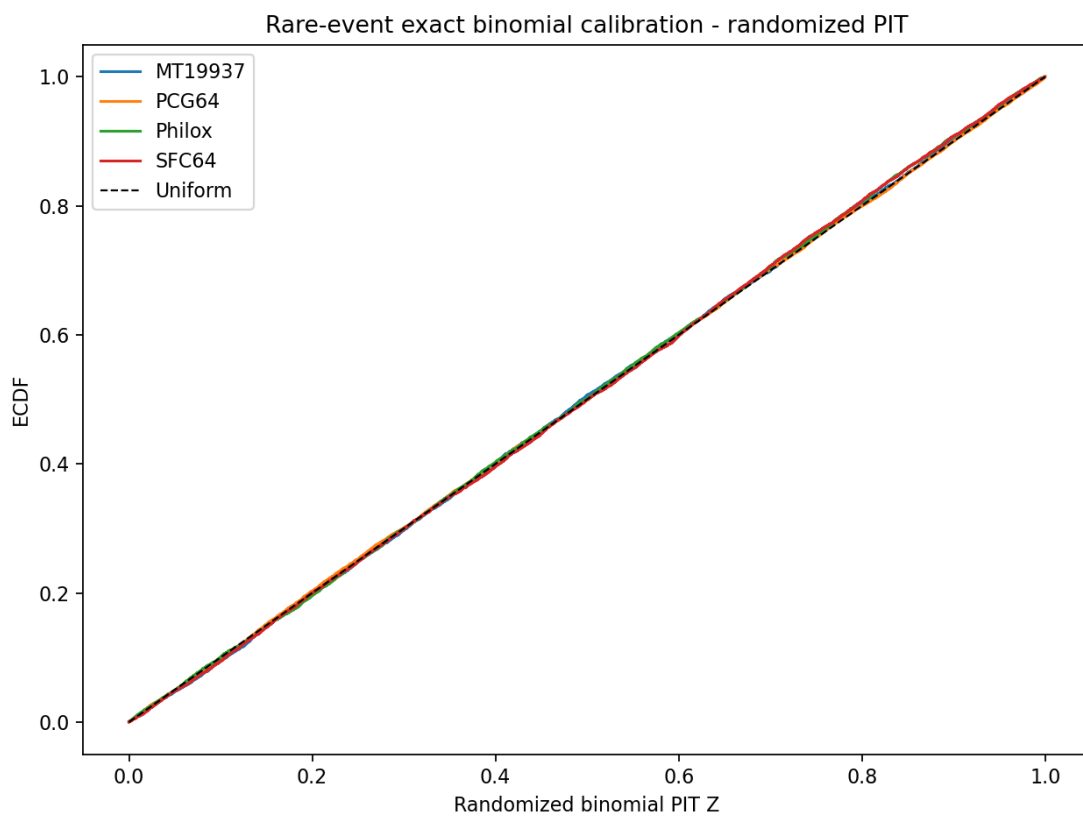


Figure 5: ECDF of randomized exact binomial PIT values for rare-event counts.

11.2 Histograms

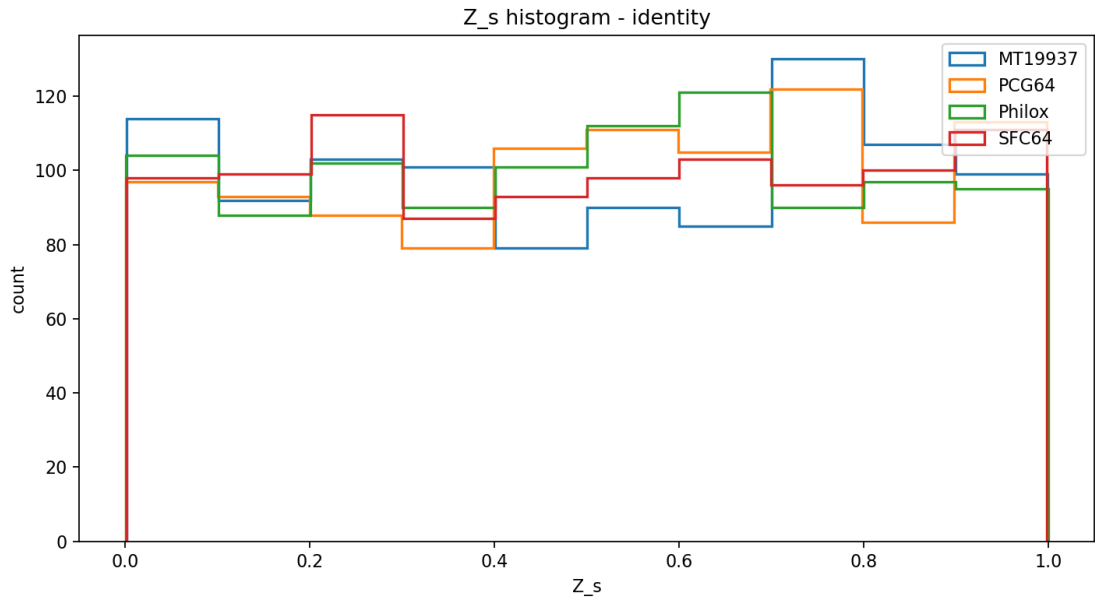


Figure 6: Histogram of Z_s for the identity integrand.

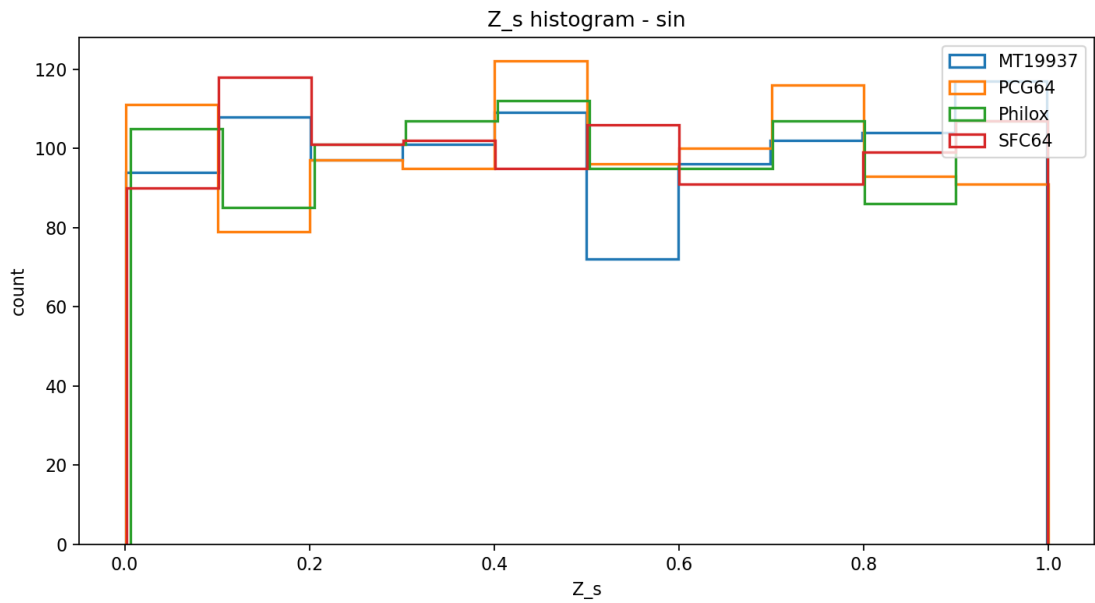


Figure 7: Histogram of Z_s for the sine integrand.

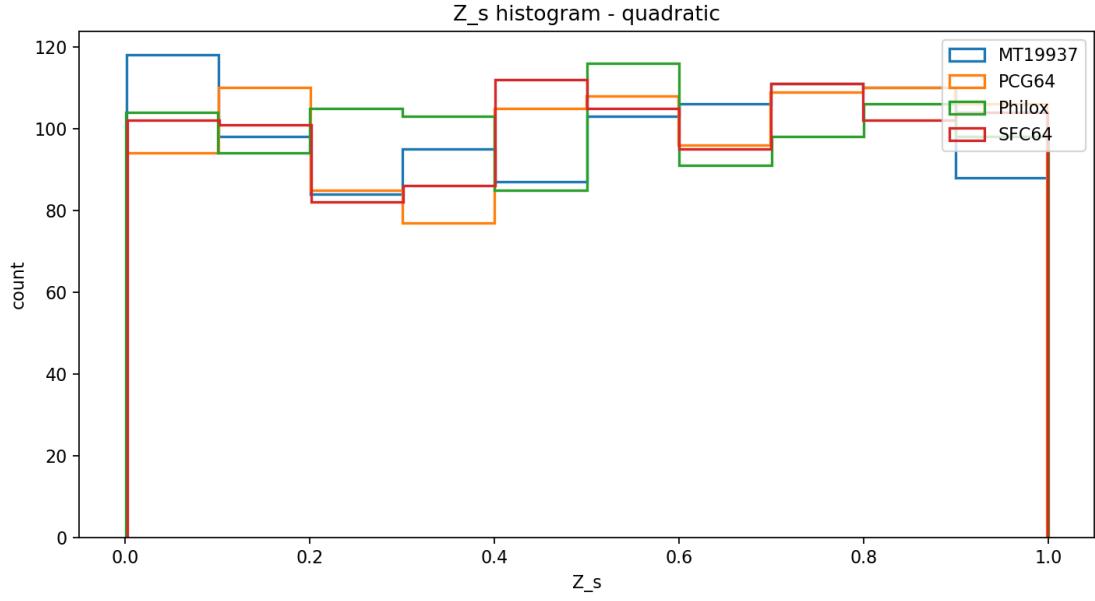


Figure 8: Histogram of Z_s for the quadratic integrand.

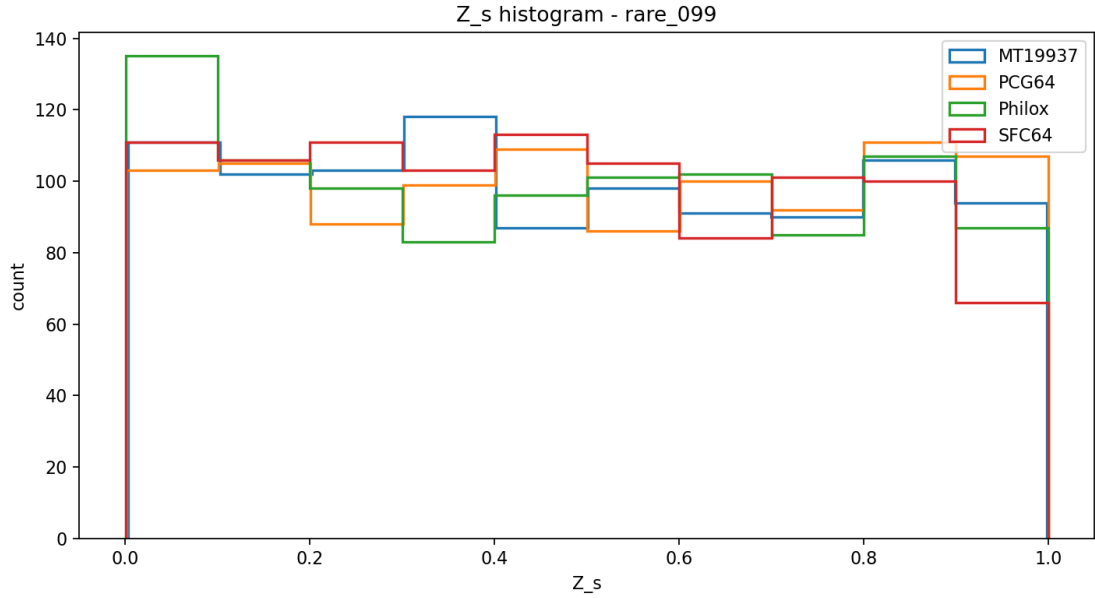


Figure 9: Histogram of Z_s for the rare-event integrand $1_{\{u>0.99\}}$.

12 Reproducibility Manifest

12.1 Public baseline bundle

```
prng-ztest/
  README.md
  requirements.txt
  scripts/
    generate_public_prng_estimates.py
    compute_z_scores.py
    plot_z_histograms.py
    plot_z_ecdf.py
  data/
    public_prng_estimates.csv
  results/
    z_scores_table.csv
    z_summary.csv
    binomial_rare_event_table.csv
    binomial_rare_event_summary.csv
  figures/
    ecdf_Zs_*.svg
    hist_Zs_*.svg
    ecdf_Zbin_rare_099.svg
    optional PNG renderings for PDF inclusion
```

The scripts, requirements file, summary table and SVG figures form the recommended versioned reproducibility bundle. The large CSV files `data/public_prng_estimates.csv` and `results/z_scores_table.csv` are generated locally by the commands below; they may be archived as release artefacts rather than committed directly.

Baseline commands:

```
python scripts/generate_public_prng_estimates.py --N 50000 --R 8 --seeds 1000
```

```
python scripts/compute_z_scores.py \
  --input data/public_prng_estimates.csv \
  --out-table results/z_scores_table.csv \
  --out-summary results/z_summary.csv
```

```
python scripts/plot_z_ecdf.py \
  --input results/z_scores_table.csv \
  --outdir figures
```

```
python scripts/plot_z_histograms.py \
  --input results/z_scores_table.csv \
  --outdir figures
```

12.2 Independent-verification requirements for reported CTACM results

For the reported Montmory_CTACM evidence to be treated as independently reproducible, the archival bundle must include:

- full SmallCrush, Crush and BigCrush logs;
- TestU01 version;
- stream adapter source or hash;

- backend implementation hashes;
- compiler flags and hardware details;
- seed and output horizon;
- SHA-256 of raw logs;
- CPU/Metal/CUDA stream hashes for the cross-backend replay claim.

13 Limitations

The present manuscript remains limited in the following ways:

- $Z_s(f)$ depends on the selected null model;
- the χ^2 calibration assumes approximately Gaussian replicated estimates;
- public CPU baseline integrands are simple diagnostics, not a full industrial payoff suite;
- no GPU public baseline is claimed in the reproducible v1.3 evidence;
- reported Montmory_CTACM results require archival raw logs and stream hashes before they can be treated as independently reproducible;
- statistical evidence does not imply cryptographic security;
- passing this protocol does not prove absence of defects outside the tested seed set, workload family or horizon.

14 Conclusion

The consolidated result is:

Raw inter-seed variance ratios can overstate seed instability.

After null calibration, the verified public CPU baseline shows:

mild calibration drift in some pairs, but no strong tail pathology.

The exact binomial rare-event PIT check is compatible for all four tested public generators.

The scientific contribution is therefore methodological:

TACM separates raw alerts from calibrated evidence.

It distinguishes compatible behaviour, calibration drift and genuine tail anomaly, while preserving the non-claims required for responsible PRNG reporting.

15 Future Work

The next experimental additions should be:

- import raw TestU01 logs and stream adapter hashes;
- run controlled injected marginal-bias tests;
- run controlled temporal-correlation tests;
- run rare-event deletion power tests;
- add backend reproducibility manifests for public generators where applicable;
- add multi- S, N, R stability grids;
- extend diagnostic integrands toward finance / insurance / HPC workloads.

References

- [1] P. L’Ecuyer and R. Simard, “TestU01: A C library for empirical testing of random number generators,” *ACM Transactions on Mathematical Software*, 33(4), 2007.
- [2] C. Doty-Humphrey, “PractRand statistical test suite,” public software documentation and source distribution.
- [3] C. R. Harris et al., “Array programming with NumPy,” *Nature*, 585, 357–362, 2020.
- [4] M. Matsumoto and T. Nishimura, “Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM TOMACS*, 1998.
- [5] M. E. O’Neill, “PCG: A family of simple fast space-efficient statistically good algorithms for random number generation,” technical report, 2014.
- [6] J. K. Salmon, M. A. Moraes, R. O. Dror and D. E. Shaw, “Parallel random numbers: As easy as 1, 2, 3,” *SC11*, 2011.
- [7] P. L’Ecuyer, “Good parameter sets for combined multiple recursive random number generators,” *Operations Research*, 47(1), 1999.